Nurarys Baibolat CS-2413

OSS Practice

Task 1

On host H10, use netstat to identify all listening TCP services and their PIDs. Determine which service is running on port 80 or 6666.

```
"Node: H10" (as superuser)
[root@secOps analyst]# netstat -tulnp | grep -E ':80|:6666'
[root@secOps analyst]# netstat -tlnp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
PID/Program name
[root@secOps analyst]#
```

Using netstat -tulnp, no listening TCP services were found on host H10. Therefore, no service is running on port **80** or **6666**.

Task 2

On R1, create a local Snort rule that generates an alert whenever any host sends an ICMP echo request to the R1 interface. Verify that the rule syntax is valid.

```
  GNU nano 6.2                                                    /etc/snort/rules/local.rules
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# ----------------
# LOCAL RULES
# ----------------
# This file intentionally does not come with signatures.  Put your local
# additions here.
alert tcp $HOME_NET any -> 209.165.202.133/32 6666 (msg: "Malicious Server Hit!"; sid:1000003;)
alert icmp any any -> $HOME_NET any (msg;"ICMP Echo Request to R1 detected"; itype:8; sid:1000001; rev:1;)
```

```
Preprocessor Object: SF_IMAP  Version 1.0  <Build 1>
Preprocessor Object: appid  Version 1.1  <Build 5>

Snort successfully validated the configuration!
Snort exiting
```

A local Snort rule was created on R1 to detect ICMP echo requests. The rule syntax was verified using snort -T, confirming the configuration was valid.

Task 3

On R1, monitor the Snort alert log in real time. From H5, ping R1 and confirm that the alert is generated.

```
"Node: H5" (as superuser)
rtt min/avg/max/mdev = 0.028/0.048/0.143/0.022 ms
[root@secOps analyst]# ping 209.165.200.225
PING 209.165.200.225 (209.165.200.225) 56(84) bytes of data.
64 bytes from 209.165.200.225: icmp_seq=269 ttl=64 time=0.372 ms
64 bytes from 209.165.200.225: icmp_seq=270 ttl=64 time=0.077 ms
64 bytes from 209.165.200.225: icmp_seq=271 ttl=64 time=0.060 ms
```

```
"Node: R1" (as superuser)
[root@secOps analyst]# tail -f /var/log/snort/alert
```

Snort alerts were monitored in real time on R1. When H5 sent ping requests to R1, alerts were generated, confirming the rule worked correctly.

Task 4

On R1, add an iptables rule in the INPUT chain to drop ICMP traffic coming only from H5. Verify that H5 cannot ping R1 while other hosts still can.

"Node: R1" (as superuser)

```
[root@secOps analyst]#
[root@secOps analyst]#
[root@secOps analyst]# iptables -A INPUT -p icmp -s 209.165.200.235 -j DROP
```

"Node: H5" (as superuser)

```
[root@secOps analyst]# ping 209.165.200.225
PING 209.165.200.225 (209.165.200.225) 56(84) bytes of data.
```

An iptables rule was configured on R1 to drop ICMP traffic from host H5 (209.165.200.235). Testing showed that H5 could no longer ping R1, while other hosts were still able to reach it.

Task 5

On H5, configure an iptables rule in the OUTPUT chain to block outgoing TCP traffic on port 80. Verify the rule by attempting to access an HTTP website using wget.

"Node: H5" (as superuser)

```
[root@secOps analyst]# iptables -A OUTPUT -p tcp --dport 80 -j DROP
[root@secOps analyst]# iptables -L OUTPUT -v -n
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

    0     0 DROP       tcp  --  *      *       0.0.0.0/0            0.0.0.0/0
         tcp dpt:80
[root@secOps analyst]#
[root@secOps analyst]#
[root@secOps analyst]# wget http://google.com
--2026-02-03 10:20:23--  http://google.com/
Resolving google.com (google.com)... failed: Temporary failure in name resolutio
n.
wget: unable to resolve host address 'google.com'
```

Task 6

On H5, use tcpdump to capture exactly 10 packets exchanged with H10. Save the capture as traffic_test.pcap.

"Node: H10" (as superuser)

```
[root@secOps analyst]# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: H10-eth0@if4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state
UP group default qlen 1000
    link/ether e2:2e:4e:be:6c:a0 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 209.165.202.133/27 brd 209.165.202.159 scope global H10-eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::e02e:4eff:febe:6ca0/64 scope link
       valid_lft forever preferred_lft forever
```

```
"Node: H10" (as superuser)
[root@secOps analyst]# ping 209.165.200.235
PING 209.165.200.235 (209.165.200.235) 56(84) bytes of data.
64 bytes from 209.165.200.235: icmp_seq=1 ttl=62 time=1.47 ms
64 bytes from 209.165.200.235: icmp_seq=2 ttl=62 time=0.084 ms
64 bytes from 209.165.200.235: icmp_seq=3 ttl=62 time=0.082 ms
64 bytes from 209.165.200.235: icmp_seq=4 ttl=62 time=0.081 ms
64 bytes from 209.165.200.235: icmp_seq=5 ttl=62 time=0.073 ms
64 bytes from 209.165.200.235: icmp_seq=6 ttl=62 time=0.082 ms
```

```
"Node: H5" (as superuser)
[root@secOps analyst]# tcpdump -i H5-eth0 -c 10 -w traffic_test.pcap
tcpdump: listening on H5-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
10 packets captured
10 packets received by filter
0 packets dropped by kernel
```

On host H5, tcpdump was used to capture network traffic exchanged with host H10. The command was configured to capture exactly 10 packets and save them to the file **traffic_test.pcap**. Traffic was generated using ICMP (ping) between the hosts, and the capture automatically stopped after reaching the specified packet count. The saved file was verified to contain the captured packets.

Task 7

Open traffic_test.pcap using tcpdump -r with the -A flag. Identify any readable strings or protocol headers in the packet payloads.

```
"Node: H5" (as superuser)
0 packets dropped by kernel
[root@secOps analyst]# tcpdump -r traffic_test.pcap -A
reading from file traffic_test.pcap, link-type EN10MB (Ethernet), snapshot length 262144
10:42:04.630647 IP 209.165.202.133 > 209.165.200.235: ICMP echo request, id 10281, seq 1,
length 64
E..T.r@.>.Qz..........t,().....i......  ..................... !"#$%&'()*+,-./01234567
10:42:04.633975 IP 209.165.200.235 > 209.165.202.133: ICMP echo reply, id 10281, seq 1, le
ngth 64
E..T....@./K..........|,().....i......  ..................... !"#$%&'()*+,-./01234567
```

The initial capture file contained no packets due to connectivity issues. After generating traffic, a new capture was created, and readable ICMP payload patterns and IP headers were observed.

Task 8

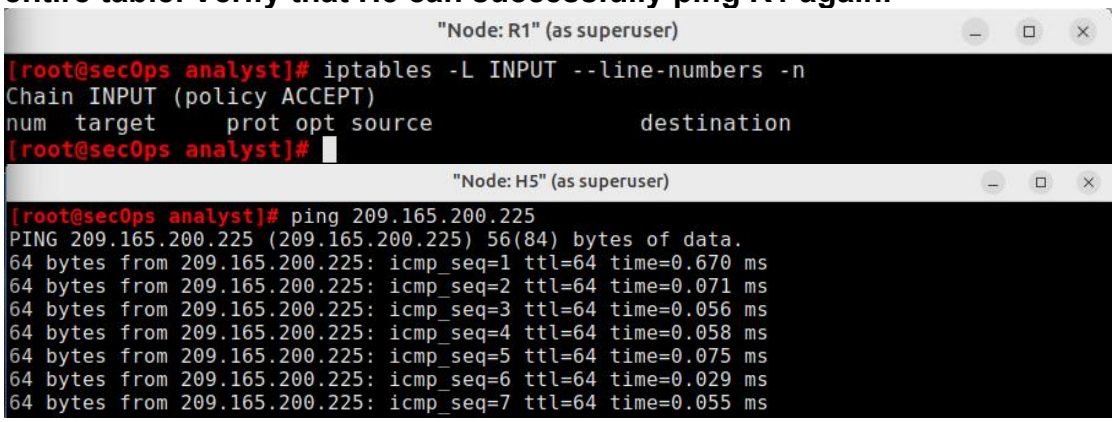
On R1, insert a rule at the top of the FORWARD chain that drops TCP traffic on port 443. Use iptables --line-numbers to verify its position.

```
"Node: R1" (as superuser)
[root@secOps analyst]# iptables -I FORWARD 1 -p tcp --dport 443 -j DROP
[root@secOps analyst]# iptables -L FORWARD --line-numbers -n
Chain FORWARD (policy ACCEPT)
num  target     prot opt source               destination
1    DROP       tcp  --  0.0.0.0/0            0.0.0.0/0            tcp dpt:443
[root@secOps analyst]# wget https://google.com
--2026-02-03 10:46:39--  https://google.com/
Resolving google.com (google.com)... failed: Temporary failure in name resolutio
n.
wget: unable to resolve_host address 'google.com'
```

An iptables rule was inserted at the top of the FORWARD chain on R1 to drop TCP traffic destined for port 443. The rule position was verified using iptables --line-numbers, confirming it was placed as rule number 1.

Task 9

Remove the ICMP firewall rule created in Task 4 without flushing the entire table. Verify that H5 can successfully ping R1 again.



The ICMP DROP rule in the INPUT chain was no longer present, as verified using iptables --line-numbers. The chain policy remained ACCEPT, and H5 successfully pinged R1, confirming the firewall rule had been removed without flushing the table.